



MEDIA INTELLIGENCE DASHBOARD

Vibecheck Cinema

TMDB + YouTube Data Pipeline

Movie data caching • sentiment analysis • resilient discovery

Browser

Pipeline

TMDB + YouTube

Why This Pipeline Exists

01

Unified Discovery

Users need a single interface for movie metadata and audience signals across multiple sources.

02

Unstructured Data

YouTube comments are high-volume and unstructured; the pipeline must extract a clear sentiment signal.

03

API Efficiency

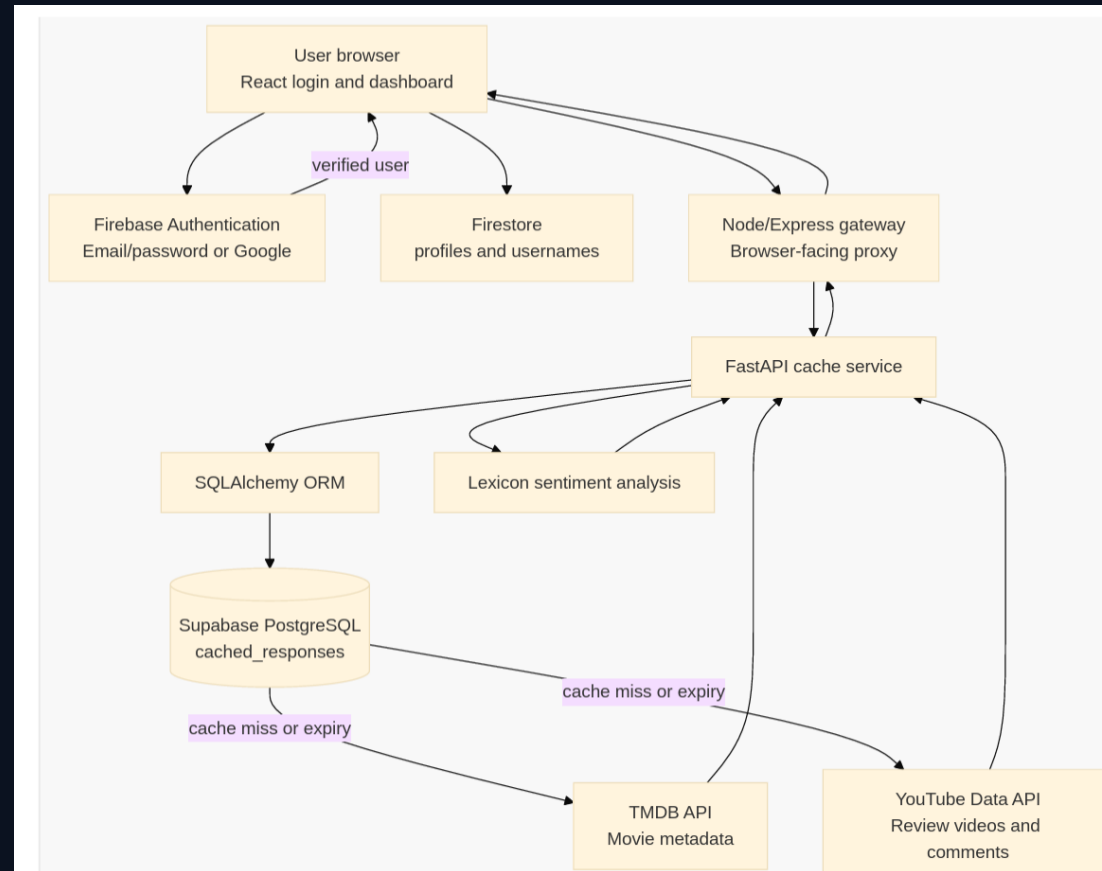
Direct provider calls for every visitor increase latency and consume TMDB/YouTube quotas.

04

Resilience

A persistent cache decouples the user experience from temporary upstream API failures.

System Flow at a Glance



Browser → Gateway → Cache Service → PostgreSQL (SQLAlchemy ORM). TMDB metadata is lazily refetched on cache miss or expiry; YouTube review data is fetched live from the cache service on every request and never stored.

TMDB Movie-Data Path

Metadata Collection

Endpoints: Popular, Now Playing, Upcoming, Top Rated, Search, Details

Normalization: Raw JSON mapped to a clean schema — ID, Title, Overview, Poster URL, Rating

Details: Extended fetch adds Credits (Director, Cast) and Videos (Trailers)

Stable cache keys drive every PostgreSQL lookup — same request, same key, same row.

CACHE KEY GENERATION

Search Key

search:batman:page=1

List Key

list:popular:page=1

Details Key

movie:27205:details

Keys are stable strings used for PostgreSQL lookups.

Refresh Intervals & Timestamp Formula

Data Category	Refresh Interval	Fetch Logic
Movie Lists (Popular, etc.)	6 Hours	TMDB called after 6h or on cache miss.
Search Results	12 Hours	Repeated searches reuse stored JSON.
Movie Details	30 Days	Metadata changes slowly; long-term cache.

```
expires_at = current_utc_time + selected_ttl
```

Lazy Refresh: The cache is not refreshed by a timer. The next request after expiry triggers a fresh API call.

Only TMDB responses are cached. YouTube review data has no TTL row — it is fetched live on every request and never written to PostgreSQL.

YouTube Review-Data Path

01

Video Search

Search for up to 5 relevant English review videos using the query: "{title} movie review."

02

Comment Collection

Request up to 40 top-level comments per video via the commentThreads endpoint.

03

Live Scoring

Comments go straight to the lexicon scorer and back to the user — nothing is written to the cache table.

RESILIENT FETCHING

If YouTube collection fails, TMDB metadata is still returned with a recorded source error.

From Comments to a Movie Signal

Lexicon-Based Scoring

A transparent algorithm counts predefined positive and negative words in review sentences.

ASPECTS SCORED

Acting

Story

Visuals

Music

Pacing

Limits: does not handle sarcasm, negation, or complex context (non-ML approach).

Verdict Thresholds

$\geq 50\%$

Mostly Positive

$< 50\%$

Mostly Negative

0

No Data / Not Scored

What Runs Where



Frontend Shell

React 19 + Vite

Manages Firebase Authentication and mounts the dashboard shell.



Cinema Dashboard

Static HTML / JS

Implements the main discovery UI, loaded via iframe in the React shell.



Application Gateway

Node / Express

Proxies requests, handles tRPC, and provides an in-memory cache fallback.



Cache Service

FastAPI + SQLAlchemy

Manages the persistent PostgreSQL cache and sentiment pipeline.

SUMMARY

Key Takeaways

01

Decoupled architecture separates auth, UI, and data services.

02

Persistent caching reduces API latency and quota consumption.

03

Lazy refresh strategy ensures data freshness without background overhead.

04

Lexicon sentiment provides immediate, interpretable audience signals.